

GUÍA PRÁCTICA

TypeORM Repository

con NestJS y SQLite

Instalación · Entidades · Repositorios · Servicios

01 / Instalación

1. TypeORM + NestJS

```
npm install --save  
@nestjs/typeorm typeorm
```

2. Driver SQLite

```
npm install sqlite3 --save
```

Otros drivers disponibles

PostgreSQL / CockroachDB	<code>npm install pg</code>
MySQL / MariaDB	<code>npm install mysql2</code>
Microsoft SQL Server	<code>npm install mssql</code>
Oracle	<code>npm install oracledb</code>
sql.js (in-memory)	<code>npm install sql.js</code>

✓ SQLite crea un archivo .db local — ideal para desarrollo y pruebas.

02 / Configuración en AppModule

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'sqlite',      // archivo local
      database: 'products.db',
      entities: [ProductEntity],
      synchronize: true,   // crea tablas automáticamente
    }),
    TypeOrmModule.forFeature([ProductEntity])
  ],
})
export class AppModule {}
```

💡 `forFeature()` registra las entidades disponibles en los módulos hijos — incluirlo en cada módulo que use el repositorio.

03 / Definición de la Entidad

```
@Entity('products')
export class ProductEntity {

    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    name: string;

    @Column()
    code: string;

    @Column({ default: 0 })
    price: number;
}
```

`@Entity('products')`

Mapea la clase a la tabla 'products' en SQLite.

`@PrimaryGeneratedColumn()`

Columna PK con autoincrement.

`@Column()`

SQLite generado:

INTEGER, TEXT, TEXT, INTEGER

04 / La interfaz que debe implementar

```
export interface ProductsRepository {  
  
  findAll():  
    Product[] | Promise<Product[]>;  
  
  findById(id: number):  
    Product | undefined  
    | Promise<Product | undefined>;  
  
  create(input: CreateProductInput):  
    Product | Promise<Product>;  
  
  update(id: number, input: UpdateProductInput):  
    Product | undefined  
    | Promise<Product | undefined>;  
  
  remove(id: number):  
    Product | undefined  
    | Promise<Product | undefined>;  
}
```

findAll()

repo.find()

findById(id)

repo.findOneBy({ id })

create(input)

repo.create(input) + save()

update(id, input)

findOneBy + Object.assign + save

remove(id)

findOneBy + repo.remove(entity)

05 / Clase que implementa la interfaz

```
@Injectable()
export class TypeOrmProductsRepository
  implements ProductsRepository {

  constructor(
    @InjectRepository(ProductEntity)
    private repo: Repository<ProductEntity>
  ) {}

  findAll() {
    return this.repo.find();
  }
  findById(id: number) {
    return this.repo.findOneBy({ id })
      .then(e => e ?? undefined);
  }
  async create(input: CreateProductInput) {
    return this.repo.save(
      this.repo.create(input));
  }
  async update(id, input) { ... }
  async remove(id) { ... }
}
```

implements ProductsRepository

TypeScript verifica en compilación que todos los métodos están implementados correctamente.

Inyección por token o clase

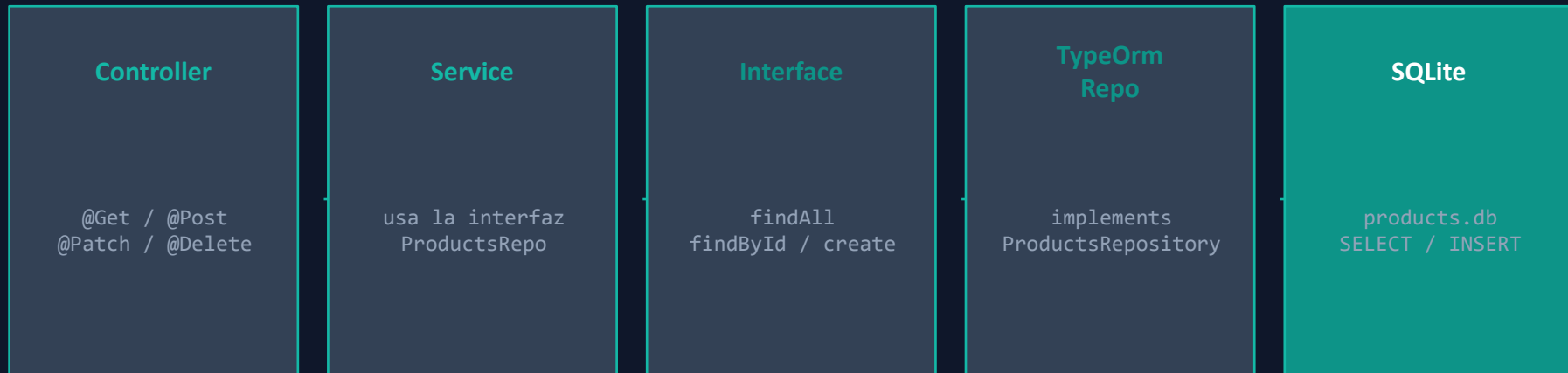
Registrá la clase como provider y usá la interfaz como tipo — desacoplás la implementación.

null → undefined

findOneBy() devuelve null; la interfaz espera undefined. Convertí siempre con ?? undefined o || undefined.

Flujo completo

Controller → Service → Interface → TypeOrmProductsRepository → SQLite



La interfaz desacopla el Service de TypeORM



null → undefined: respetar el contrato de la interfaz



Fácil de testear con un mock que respete la interfaz



synchronize: true crea la tabla automáticamente